

BÀI THỰC HÀNH SỐ 1

I. Mục tiêu

Sau khi hoàn thành bài Lab, sinh viên có thể:

Kiến thức

Hiểu vai trò và tầm quan trọng của tiền xử lý dữ liệu trong quy trình khai phá dữ liệu. Nhận biết được các loại đối tượng dữ liệu và các kiểu thuộc tính. Nắm được các kỹ thuật tiền xử lý phổ biến như:

1. Làm sạch dữ liệu
2. Tích hợp dữ liệu
3. Thu giảm dữ liệu
4. Biến đổi dữ liệu
5. Rời rạc hóa dữ liệu

Kỹ năng

- Xác định các vấn đề thường gặp trong dữ liệu như: thiếu dữ liệu, nhiễu, trùng lặp, không nhất quán.
- Lựa chọn và áp dụng các phương pháp tiền xử lý phù hợp với từng loại dữ liệu.
- Sử dụng công cụ Python để thực hiện các bước tiền xử lý.
- Chuẩn bị dữ liệu đầu vào phù hợp cho các thuật toán khai phá dữ liệu.

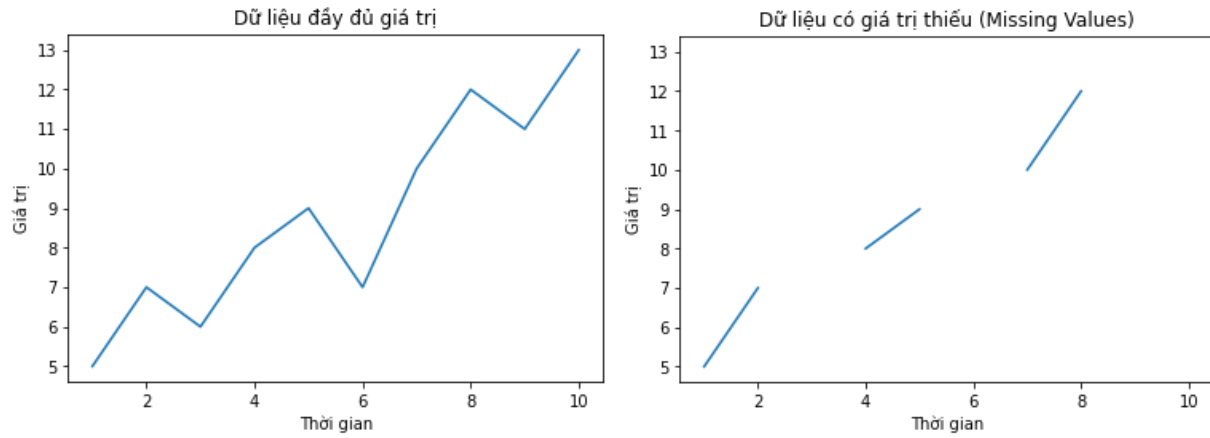
II. Tóm tắt lý thuyết

1. Giới thiệu về tiền xử lý dữ liệu

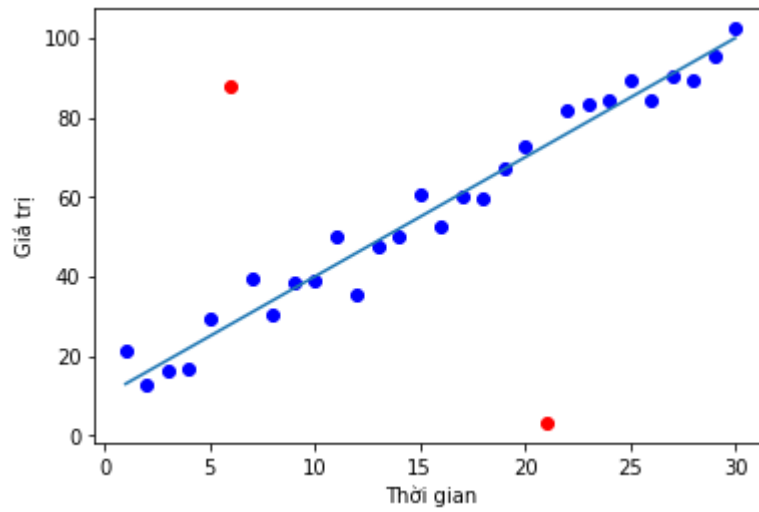
Tiền xử lý dữ liệu (Data Preprocessing) là bước chuẩn bị dữ liệu trước khi áp dụng các thuật toán khai phá dữ liệu hoặc học máy.

Dữ liệu thực tế thường:

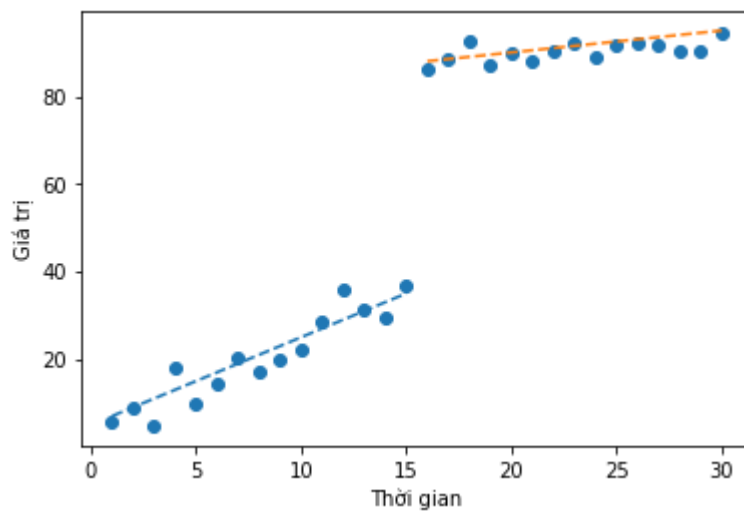
- Thiếu giá trị (missing values)



- Có nhiễu (noise)



- Không nhất quán (Inconsistent)



Mục tiêu:

- Nâng cao chất lượng dữ liệu
- Giảm kích thước dữ liệu
- Tăng hiệu quả và độ chính xác của mô hình

2. Đối tượng dữ liệu và các loại thuộc tính

Đối tượng dữ liệu (Data objects): là các bản ghi, mẫu hoặc dòng dữ liệu.

Các loại thuộc tính:

Danh mục (Nominal): Không có thứ tự

Ví dụ: giới tính, màu sắc

Thứ bậc (Ordinal): Có thứ tự nhưng không có khoảng cách xác định

Ví dụ: mức độ hài lòng (thấp, trung bình, cao)

Khoảng (Interval): Có thứ tự và khoảng cách, không có mốc 0 tuyệt đối

Ví dụ: nhiệt độ (°C)

Tỷ lệ (Ratio): Có mốc 0 tuyệt đối

Ví dụ: tuổi, thu nhập, chiều cao

3. Làm sạch dữ liệu (Data Cleaning)

Mục tiêu: xử lý dữ liệu thiếu, nhiễu hoặc không nhất quán.

Các phương pháp có thể áp dụng bao gồm

3.1. Xóa bản ghi:

Ví dụ: Trong bảng dữ liệu sinh viên có 100 dòng, có 3 sinh viên bị thiếu điểm Toán. Nếu số lượng thiếu rất ít, ta có thể xóa 3 dòng đó để đảm bảo dữ liệu sạch.

Áp dụng khi: Tỷ lệ thiếu nhỏ và việc xóa không làm mất nhiều thông tin quan trọng

3.2. Thay bằng giá trị trung bình/trung vị

Ví dụ: Một bộ dữ liệu về thu nhập có 1 người bị thiếu thông tin.

Ta có thể tính: Trung bình thu nhập của những người còn lại = 15 triệu. Điền 15 triệu vào chỗ bị thiếu. Nếu dữ liệu bị lệch (có người thu nhập quá cao), ta dùng trung vị thay vì trung bình để tránh bị ảnh hưởng bởi ngoại lai.

Áp dụng khi:

Muốn giữ lại số lượng dữ liệu

Dữ liệu phân bố tương đối ổn định

3.3. Nội suy hoặc dự đoán

Ví dụ: Dữ liệu nhiệt độ theo ngày:

Ngày 1: 30°C

Ngày 2: ?

Ngày 3: 34°C

Ta có thể nội suy: Ngày 2 $\approx 32^{\circ}\text{C}$ hoặc dùng mô hình hồi quy để dự đoán giá trị thiếu dựa trên các biến khác.

Áp dụng khi:

Dữ liệu có tính liên tục theo thời gian

Có mối quan hệ rõ ràng giữa các biến

3.4. Làm trơn dữ liệu nhiễu:

Dùng Binning

Ví dụ:

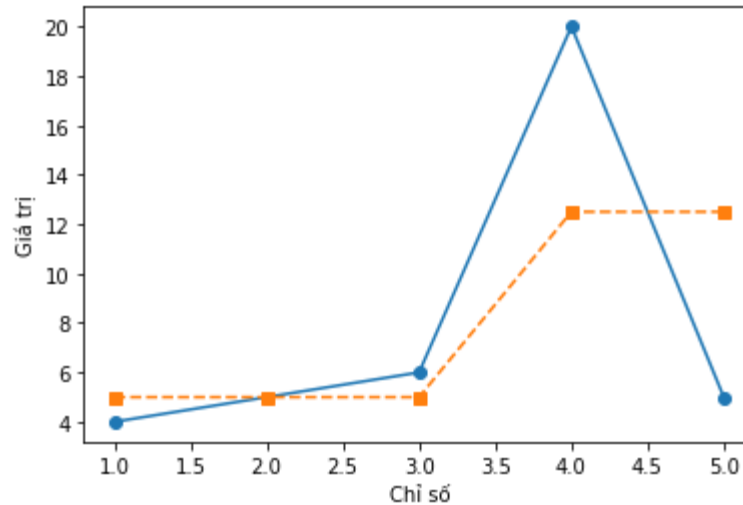
Điểm kiểm tra: 4, 5, 6, 20, 5

Ta chia thành nhóm (bin) 3 giá trị:

Nhóm 1: 4, 5, 6 $\rightarrow$ thay bằng 5

Nhóm 2: 20, 5 $\rightarrow$ thay bằng 12.5

$\Rightarrow$ Giúp giảm ảnh hưởng dao động nhỏ.

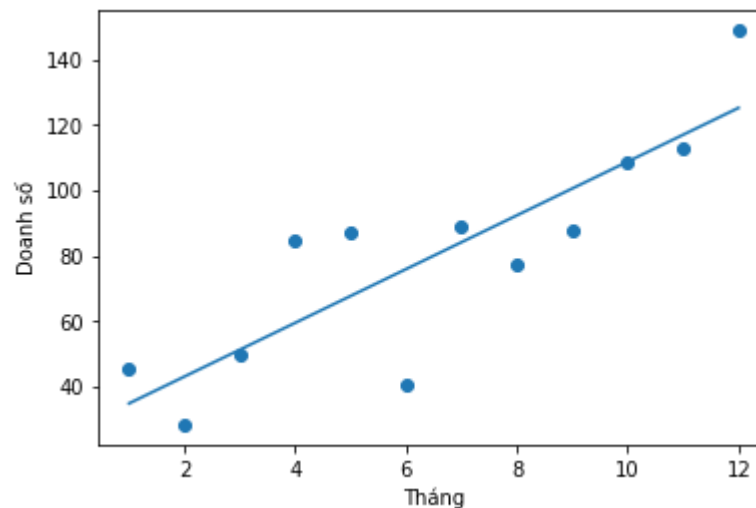


Binning giúp làm mượt dữ liệu bằng cách gom nhóm.

Dùng Regression

Ví dụ: Doanh số theo tháng có dao động mạnh.

Ta dùng hồi quy tuyến tính để tìm đường xu hướng chung thay vì từng điểm nhiễu riêng lẻ.



Regression giúp:

- Nhìn rõ xu hướng dài hạn
- Giảm ảnh hưởng nhiễu ngẫu nhiên

3.5. Phát hiện và loại bỏ ngoại lai (outliers)

Ví dụ: Lương nhân viên: 10 triệu, 12 triệu, 11 triệu, 9 triệu, 500 triệu

Giá trị 500 triệu quá khác biệt so với phần còn lại.

Có thể:

- Kiểm tra lại xem có nhập sai không
- Nếu là lỗi nhập liệu → loại bỏ
- Nếu là dữ liệu thật → cần xem xét riêng

Có thể phát hiện bằng:

- Z-score
- IQR (khoảng tứ phân vị)
- Biểu đồ boxplot

4. Tích hợp dữ liệu (Data Integration)

Kết hợp dữ liệu từ nhiều nguồn khác nhau.

Vấn đề thường gặp:

- Trùng lặp dữ liệu
- Không nhất quán về đơn vị hoặc định dạng
- Xung đột giá trị

Giải pháp:

- Loại bỏ trùng lặp
- Chuẩn hóa định dạng
- Hợp nhất các bảng theo khóa chung

Ví dụ

Một công ty bán hàng thu thập dữ liệu từ 2 nguồn khác nhau:

Nguồn 1 (Hệ thống bán hàng)

Mã KH	Doanh thu (VND)
A01	5.000.000
A02	7.000.000
A03	6.000.000

Nguồn 2 (Hệ thống marketing)

Mã KH	Doanh thu (triệu VND)
A02	7
A03	6
A04	8

Vấn đề phát sinh:

1. Trùng lặp dữ liệu: A02, A03 xuất hiện ở cả hai bảng
2. Không nhất quán về đơn vị: Bảng 1: VND, Bảng 2: triệu VND
3. Xung đột giá trị: Nếu A02 ở 2 bảng có giá trị khác nhau. Cần kiểm tra lại.

Có thể xử lý bằng cách:

- Chuẩn hóa đơn vị (đổi tất cả về VND)
- Hợp nhất theo khóa chung (Mã KH)
- Loại bỏ trùng lặp

5. Thu giảm dữ liệu (Data Reduction)

Mục tiêu: giảm kích thước dữ liệu nhưng vẫn giữ được thông tin quan trọng.

Các phương pháp:

- Giảm số thuộc tính (Feature Selection)
- Giảm số bản ghi (Sampling)
- Nén dữ liệu
- Giảm chiều (PCA)

Lợi ích:

- Tăng tốc độ xử lý
- Giảm chi phí lưu trữ
- Giảm nguy cơ overfitting

6. Biến đổi dữ liệu (Data Transformation)

Chuyển đổi dữ liệu về dạng phù hợp cho mô hình.

Các kỹ thuật:

6.1. Chuẩn hóa (Normalization)

Dữ liệu khách hàng có:

- Tuổi: 18 – 60
- Thu nhập: 5 triệu – 200 triệu

Nếu khoảng giá trị chênh lệch lớn, mô hình sẽ bị lệch về biến có giá trị lớn.

Ta có thể đưa dữ liệu về khoảng $[0,1]$

Công thức:

$$x' = \frac{x - \min}{\max - \min}$$

Ví dụ: Tuổi 18–60 có thể chuyển thành 0–1

Thu nhập 5–200 triệu có thể chuyển thành 0–1

Chuẩn hóa theo trung bình và độ lệch chuẩn:

$$x' = \frac{x - \mu}{\sigma}$$

Phù hợp cho các mô hình như:

- Logistic Regression
- SVM
- KNN

6.2. Mã hóa dữ liệu phân loại (Encoding)

Ví dụ

Biến Giới tính:

- Nam
- Nữ

Một số các mô hình hay phương pháp không hiểu nếu đầu vào là chữ nên cần chuyển thành số.

Label Encoding

Nam = 0

Nữ = 1

One-hot Encoding

Nam chuyển thành (1,0)

Nữ chuyển thành (0,1)

Phù hợp khi không có thứ tự giữa các giá trị.

6.3. Tạo thuộc tính mới (Feature engineering)

Ví dụ: Dữ liệu thương mại điện tử

Giả sử có bảng dữ liệu:

Ma_KH	So_luong	Gia_sp	Ngày_dat_hang
A01	2	150000	1/5/2024
A02	1	500000	2/10/2024
A03	5	120000	3/15/2024

Ta có thể tạo thuộc tính mới như sau:

1. Tổng tiền đơn hàng

Tạo thuộc tính giúp mô hình hiểu giá trị thực tế của đơn hàng.

$$Tong_tien = So_luong \times Gia_sp$$

2. Tháng đặt hàng

Từ Ngày_dat_hang, tách thành: Tháng, Quý, Ngày trong tuần giúp phân tích hành vi mua theo mùa vụ.

3. Nhóm giá sản phẩm

Phân loại: Giúp mô hình học theo nhóm thay vì số thô. Ví dụ:

- $< 200.000 \rightarrow$ Giá thấp
- $200.000 - 400.000 \rightarrow$ Trung bình
- $400.000 \rightarrow$ Cao

7. Rời rạc hóa dữ liệu (Data Discretization)

Chuyển dữ liệu liên tục thành các khoảng giá trị rời rạc.

Ví dụ:

- Tuổi $\rightarrow (0-18), (19-35), (36-60), (>60)$

Phương pháp:

- Equal-width binning
- Equal-frequency binning

- Dựa trên cây quyết định hoặc entropy

III. Cài đặt môi trường

1. Sử dụng Anaconda

Truy cập: <https://www.anaconda.com/products/distribution>

Tải phiên bản phù hợp hệ điều hành

2. Sử dụng Python + pip

Truy cập: <https://www.python.org/downloads/>

Tải và cài Python (nhớ chọn Add to PATH)

3. Sử dụng Google Colab hoặc Kaggle

Google Colab

Truy cập: <https://colab.research.google.com>

Đăng nhập bằng tài khoản Google

Chọn New Notebook

Kaggle

Truy cập: <https://www.kaggle.com>

Đăng nhập sau đó chọn **Code** và **New Notebook**

IV. Thực hành

Bài tập 1. Cho file *bank.csv* với các thuộc tính được mô tả như sau:

age: Tuổi khách hàng

job: Nghề nghiệp

marital: Tình trạng hôn nhân

education: Trình độ học vấn

default: Có nợ xấu không (yes/no)

balance: Số dư tài khoản

housing: Có vay mua nhà không (yes/no)

loan: Có vay cá nhân không (yes/no)

contact: Phương thức liên hệ

day: Ngày gọi trong tháng

month: Tháng gọi

duration: Thời lượng cuộc gọi (giây)

campaign: Số lần liên hệ trong chiến dịch hiện tại

pdays: Số ngày từ lần liên hệ trước (-1 = chưa từng liên hệ)

previous: Số lần liên hệ trước đó

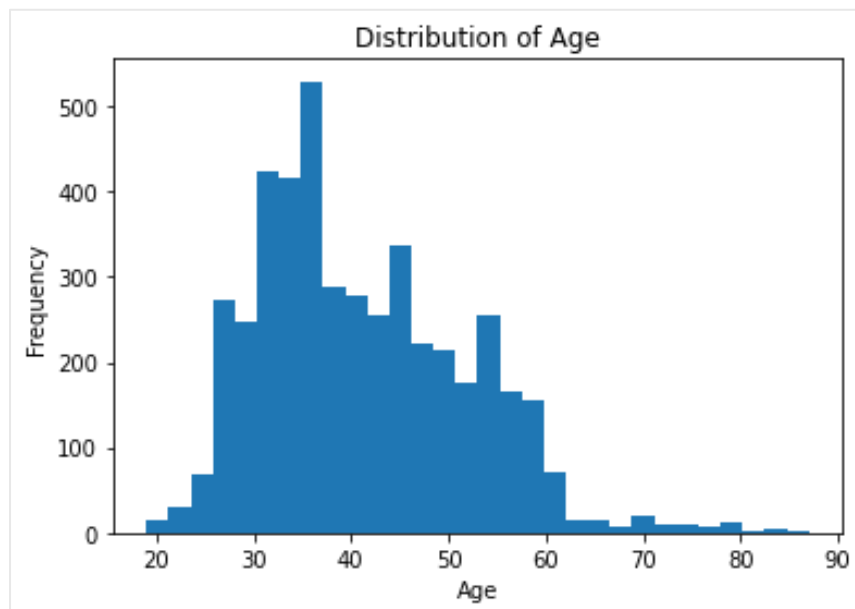
poutcome: Kết quả chiến dịch trước

y: Khách hàng có đăng ký tiền gửi hay không (biến mục tiêu)

Sử dụng file *bank.csv* trên và ngôn ngữ lập trình Python, thực hiện các yêu cầu sau:

Phân bố dữ liệu

1. Vẽ histogram cho biến age. Khách hàng tập trung ở độ tuổi nào?



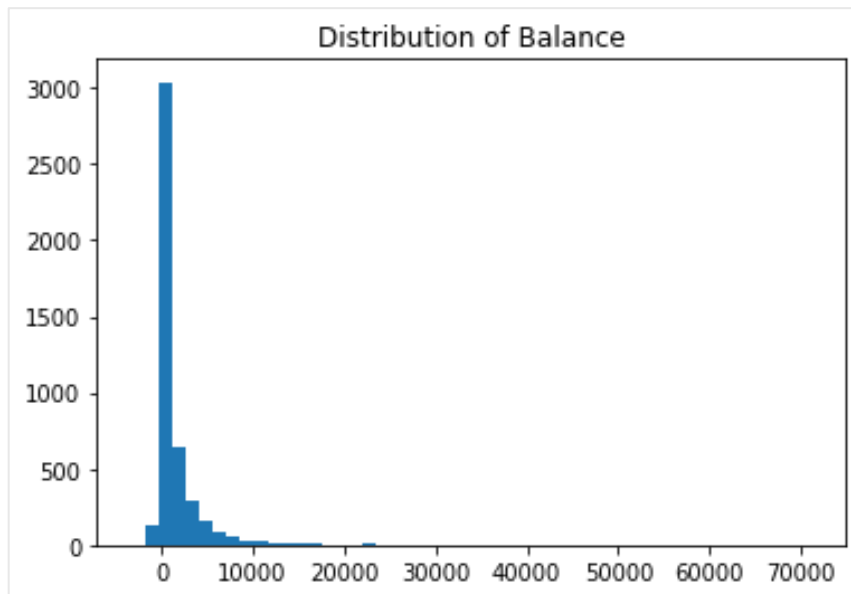
Gợi ý: Có thể dùng

```
plt.hist(df['age'], bins=30)
```

```
df['age'].plot(kind='hist')
```

```
df['age'].describe()
```

2. Vẽ histogram cho balance. Có nhiều giá trị ngoại lai không ?



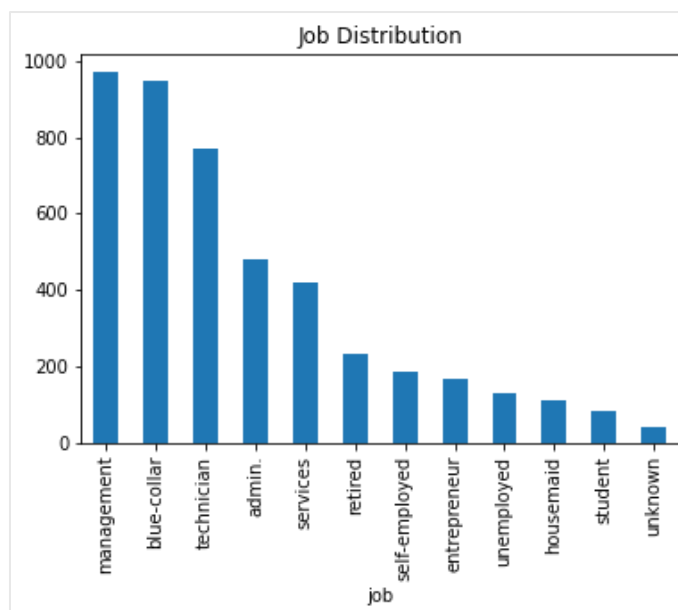
Gợi ý: Có thể dùng

```
plt.hist(df['balance'], bins=50)
```

```
plt.boxplot(df['balance'])
```

Dùng IQR: `df['balance'].quantile()`

3. Vẽ biểu đồ cột cho job. Nghề nào chiếm tỷ lệ cao nhất ?

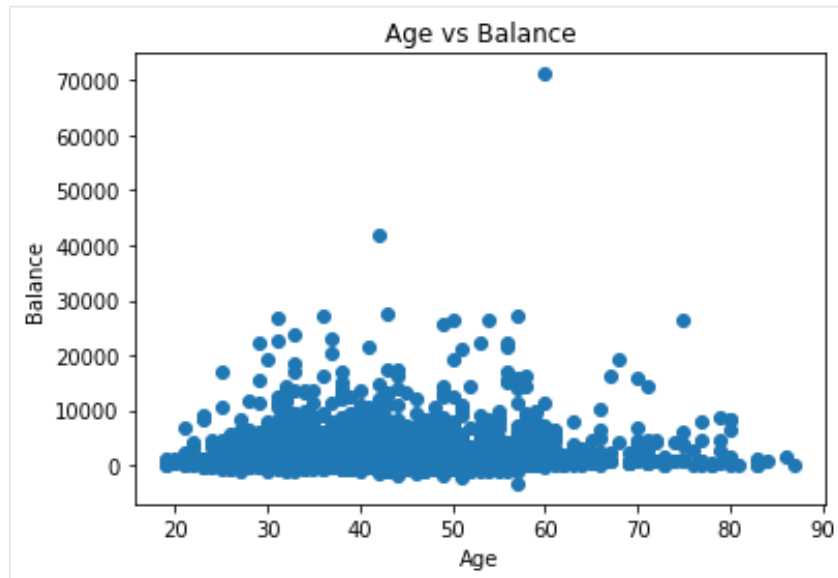


Gợi ý: Có thể dùng

```
df['job'].value_counts().plot(kind='bar')  
  
value_counts(normalize=True)
```

Quan hệ giữa các biến

4. Vẽ scatter plot giữa balance và age. Có mối liên hệ nào hay không?

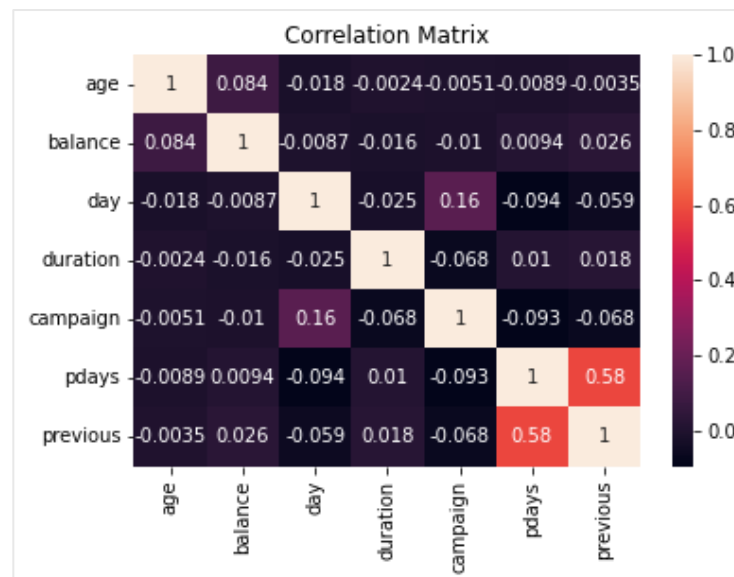


Gợi ý: Có thể dùng

```
plt.scatter(df['age'], df['balance'])
```

Tính tương quan: `df[['age', 'balance']].corr()`

5. Vẽ heatmap ma trận tương quan giữa các thuộc tính.



Gợi ý: Có thể dùng

```
df.corr(numeric_only=True)
```

Vẽ heatmap bằng `sns.heatmap()`

6. Số lần liên hệ (**campaign**) ảnh hưởng thế nào đến kết quả y ?

Gợi ý: Có thể dùng

```
pd.crosstab(df['campaign'], df['y'], normalize='index')
```

```
Boxplot: sns.boxplot(x='y', y='campaign', data=df)
```

Phân tích theo thời gian

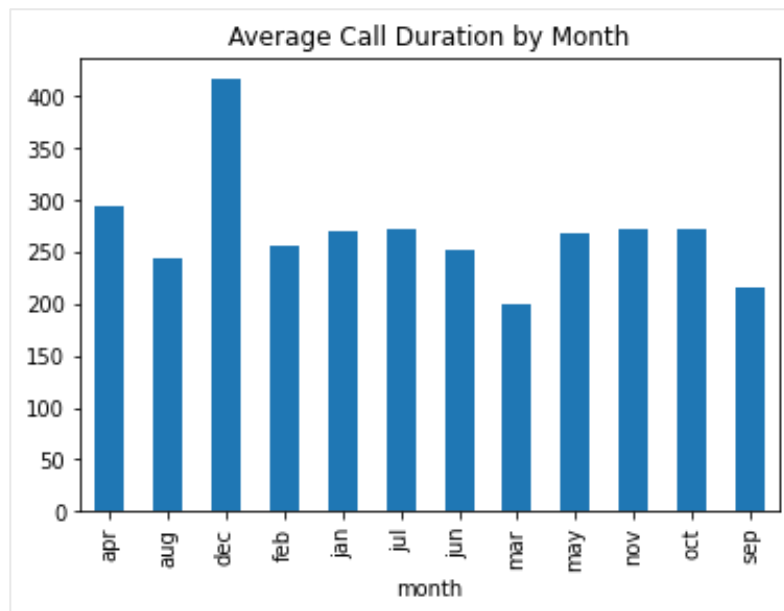
7. Tháng nào có tỷ lệ đăng ký cao nhất?

Gợi ý: Có thể dùng

```
pd.crosstab(df['month'], df['y'], normalize='index')
```

```
Sắp xếp: .sort_values(by='yes', ascending=False)
```

8. Thời lượng cuộc gọi trung bình theo từng tháng.



Gợi ý: Có thể dùng

```
df.groupby('month')['duration'].mean()
```

Vẽ: `.plot(kind='bar')`

Khám phá dữ liệu (Data Understanding)

9. Bộ dữ liệu có bao nhiêu bản ghi và bao nhiêu thuộc tính?

Gợi ý: Có thể dùng

`df.shape`

`df.info()`

10. Kiểm tra xem có giá trị thiếu (missing values) hay không?

Gợi ý: Có thể dùng

`df.isnull().sum()`

Kiểm tra "unknown": `df.eq('unknown').sum()`

11. Biến mục tiêu (target variable) là gì?

12. Có thuộc tính nào có giá trị âm bất thường không (ví dụ: balance = -88)?

Gợi ý: Có thể dùng

`df[df['balance'] < 0]`

`df.describe()`

Làm sạch dữ liệu (Data Cleaning)

13. Kiểm tra và xử lý các giá trị: pdays = -1 có ý nghĩa gì? Có nên thay thế hoặc chuyển thành NaN hay không?

Gợi ý: Có thể dùng

Kiểm tra số lượng: `df['pdays'].value_counts()`

Nếu thay bằng NaN: `df['pdays'].replace(-1, np.nan)`

14. Xử lý các giá trị "unknown" trong các cột như: contact, poutcome

Gợi ý: Có thể dùng

`df.replace('unknown', np.nan)`

Thay bằng mode: `df.fillna(df.mode().iloc[0])`

15. Kiểm tra trùng lặp bản ghi và loại bỏ nếu có.

Gợi ý: Có thể dùng

df.duplicated().sum()

Xóa: *df.drop_duplicates()*

16. Kiểm tra và xử lý giá trị ngoại lai trong: balance, duration

Gợi ý: Có thể dùng

Boxplot: *sns.boxplot()*

IQR method: *quantile(0.25)*, *quantile(0.75)*

Chuyển đổi dữ liệu (Transformation)

17. Chuẩn hóa các biến số: age, balance, duration

Gợi ý: Có thể dùng

from sklearn.preprocessing import StandardScaler

MinMaxScaler()

scaler.fit_transform()

18. Mã hóa các biến phân loại: job, marital, education, housing, loan. Nên dùng Label Encoding hay One-hot Encoding cho các biến phân loại này ? Vì sao ?

Gợi ý: Có thể dùng

One-hot encoding: *pd.get_dummies()*

Label encoding: *LabelEncoder()*

19. Chuyển đổi biến month thành dạng số (1 - 12).

Gợi ý:

Tạo dictionary mapping

Dùng *df['month'].map(dict)*

Feature Engineering

20. Tạo biến mới: Nhóm tuổi (young, middle, senior), Tổng số lần liên hệ (campaign + previous) và biến nhị phân: có từng được liên hệ trước đó hay chưa

Gợi ý: Có thể dùng

Nhóm tuổi: *pd.cut()*

Tổng liên hệ: $df['campaign'] + df['previous']$

Biến nhị phân: $np.where()$

21. Tạo biến: $duration_per_campaign = duration / campaign$

Gợi ý: Có thể dùng

$df['duration'] / df['campaign']$

Phân tích mất cân bằng dữ liệu

22. Tỷ lệ giữa $y = \text{yes}$ và $y = \text{no}$ là bao nhiêu ?

Gợi ý: Có thể dùng

$df['y'].value_counts(normalize=True)$

23. Nếu dữ liệu mất cân bằng, đề xuất phương pháp xử lý (undersampling, oversampling, SMOTE).

Gợi ý: Có thể dùng

Oversampling: $RandomOverSampler()$

Undersampling: $RandomUnderSampler()$

SMOTE: $from imblearn.over_sampling import SMOTE$

Bài tập 2. Cho file *titanic.csv* với các thuộc tính được mô tả như sau:

pclass: Hạng vé (1 = cao cấp, 2 = trung cấp, 3 = phổ thông)

survived: Sống sót hay không (1 = sống, 0 = chết)

name: Họ tên hành khách

sex: Giới tính (male/female)

age: Tuổi

sibsp: Số anh chị em/vợ chồng đi cùng

parch: Số cha mẹ/con cái đi cùng

ticket: Số vé

fare: Giá vé

cabin: Số cabin

embarked: Cảng lên tàu: C = Cherbourg, Q = Queenstown, S = Southampton

boat: Số thuyền cứu sinh (nếu sống sót)

body: Số nhận dạng thi thể (nếu tử vong)

home.dest: Nơi ở cuối cùng của hành khách

Sử dụng file **titanic.csv** trên và ngôn ngữ lập trình Python, thực hiện các yêu cầu sau:

Phân bố cơ bản

1. Tỷ lệ sống sót (survived) là bao nhiêu?

Gợi ý: Có thể dùng

```
df['survived'].value_counts(normalize=True)
```

2. Phân bố giới tính (sex).

Gợi ý: Có thể dùng

```
df['sex'].value_counts()
```

3. Phân bố hạng vé (pclass).

Gợi ý: Có thể dùng

```
df['pclass'].value_counts()
```

4. Phân bố tuổi (age).

Gợi ý: Có thể dùng

```
plt.hist(df['age'].dropna())
```

```
df['age'].describe()
```

So sánh theo sống sót

5. Tỷ lệ sống sót theo giới tính?

Gợi ý: Có thể dùng

```
pd.crosstab(df['sex'], df['survived'], normalize='index')
```

6. Tỷ lệ sống sót theo hạng vé (pclass) ?

Gợi ý: Có thể dùng

```
pd.crosstab(df['pclass'], df['survived'], normalize='index')
```

7. Tỷ lệ sống sót theo nhóm tuổi ?

Gợi ý: Có thể dùng

`pd.cut()` để chia nhóm

Sau đó dùng `crosstab()`

8. So sánh giá vé (fare) giữa người sống và không sống bằng boxplot.

Gợi ý: Có thể dùng

`sns.boxplot(x='survived', y='fare', data=df)`

Khám phá dữ liệu (Data Understanding)

9. Bộ dữ liệu có bao nhiêu dòng và bao nhiêu cột ?

Gợi ý: Có thể dùng

`df.shape`

10. Ý nghĩa của từng biến là gì (pclass, sibsp, parch, boat, body...) ?

Gợi ý: Có thể dùng

`df.head()`

`df.info()`

11. Kiểm tra tỷ lệ sống sót (survived) có mất cân bằng không ?

Làm sạch dữ liệu (Data Cleaning)

12. Kiểm tra các cột có giá trị thiếu (missing values): age, cabin, embarked, boat, body, home.dest

Gợi ý: Có thể dùng

`df['survived'].value_counts(normalize=True)`

13. Có nên loại bỏ các cột: name, ticket, boat, body. Vì sao?

Gợi ý: Có thể dùng

`df.drop(columns=[...])`

Xem correlation hoặc phân tích leakage

14. Xử lý giá trị thiếu ở: age (điền trung bình/trung vị hay theo nhóm giới tính + hạng vé ?), embarked (điền mode ?)

Gợi ý: Có thể dùng

`df.groupby(['sex', 'pclass'])['age'].transform('median')`

Hoặc `fillna(df['age'].median())`

15. Kiểm tra và xử lý trùng lặp dữ liệu.

Gợi ý: Có thể dùng

`df.groupby(['sex', 'pclass'])['age'].transform('median')`

Hoặc `fillna(df['age'].median())`

Chuyển đổi dữ liệu (Transformation)

16. Mã hóa các biến phân loại: sex, embarke, pclass

Gợi ý: Có thể dùng

`pd.get_dummies()`

Hoặc `LabelEncoder()`

17. Có nên chuẩn hóa các biến: age, fare hay không ? Khi nào cần chuẩn hóa?

Gợi ý: Có thể dùng

Dùng `StandardScaler()`

18. Trích xuất danh xưng (Title) từ cột name (Mr, Mrs, Miss, Master...). Có nên dùng các danh xưng này làm feature mới không?

Gợi ý: Có thể dùng

`df['name'].str.extract()`

`value_counts()` để xem nhóm

Feature Engineering

19. Tạo biến family_size:

`family_size = sibsp + parch + 1`

`is_alone` (đi một mình hay không)

Gợi ý: Có thể dùng

`df['sibsp'] + df['parch'] + 1`

Tạo `is_alone` bằng `np.where()`

20. Tạo nhóm tuổi: trẻ em, người lớn, người cao tuổi

Gợi ý: Có thể dùng

`pd.cut()`

21. Tạo biến: `cabin_known` (có thông tin cabin hay không)

Gợi ý: Có thể dùng

`df['cabin'].notnull()`

`np.where()`

22. Tạo biến `fare_per_person`:

`fare_per_person = fare / family_size`

Gợi ý: Có thể dùng

`df['fare'] / df['family_size']`

Phân tích ngoại lai, nhiễu (Outliers)

23. Kiểm tra ngoại lai trong: fare, age

Gợi ý: Có thể dùng

`sns.boxplot()`

Hoặc IQR

24. Có nên xử lý fare = 0 không?

Gợi ý: Có thể dùng

`df[df['fare']==0]`

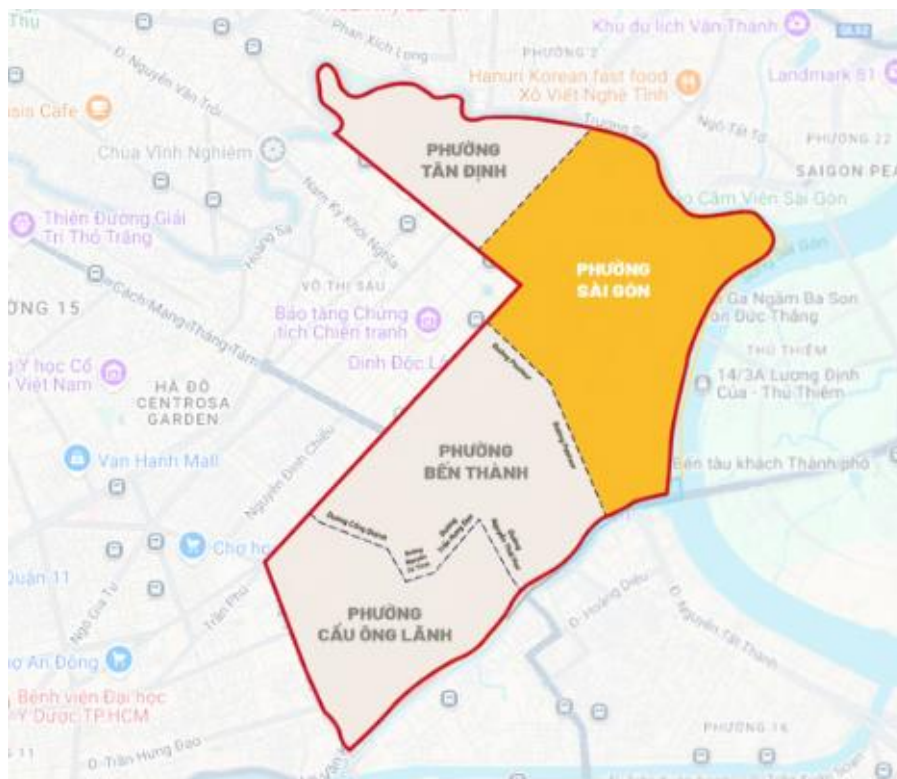
Kiểm tra tỷ lệ

Bài tập 3. Cho file *BDS_TPHCM.csv* là một file lưu trữ thông tin liên quan đến các bất động sản được rao bán ở Thành phố Hồ Chí Minh, tuy nhiên trong dữ liệu có rất nhiều các lỗi dữ liệu, nhiều, thiếu nhất quán,...

Yêu cầu 1: Thành phố Hồ Chí Minh sau khi sáp nhập đã có sự thay đổi về địa chỉ, các cấp xã, phường, hãy sử dụng cột *addressLocality* và triển khai code để tìm ra địa chỉ cũ của các bất động sản. (Có thể đối chiếu với cột *address_old* để **tham khảo**)

Ví dụ:

Với Quận 1, sau sáp nhập được tách thành 4 phường gồm: phường Tân Định, phường Sài Gòn, phường Bến Thành, phường Cầu Ông Lãnh.



Yêu cầu 2: Hãy làm sạch file dữ liệu trên tập trung vào các cột thuộc tính *Chiều dài*, *Chiều ngang*, *price_value*, *area*.

Sau đó hãy tính toán **trung bình giá VNĐ/m²** cho từng khu vực theo:

Xã, phường mới (sau sáp nhập)

Quận cũ (trước sáp nhập)

So sánh giá của một số khu vực gần nhau (trong cùng Quận cũ), phân tích sự hợp lý cho giá cả của các khu vực đó. Vẽ biểu đồ trực quan giá theo từng khu vực.

Có thể tham khảo các gợi ý thực hiện sau để thực hiện hoặc tự triển khai:

Phân tích ngoại lai (Outliers)

1. Phát hiện bất động sản có giá quá cao hoặc quá thấp.
2. Phát hiện diện tích bất thường (ví dụ 12m² nhưng giá 192 tỷ).

Trực quan hóa dữ liệu

3. Vẽ histogram phân bố giá (price_value).
4. Vẽ histogram phân bố diện tích (area).
5. So sánh giá trung bình theo từng xã (addressLocality).
6. So sánh giá/m² theo từng loại BĐS.
7. Vẽ scatter plot giữa diện tích và giá.
8. Vẽ boxplot giá theo loại BĐS.

Khám phá dữ liệu (Data Understanding)

9. Bộ dữ liệu có bao nhiêu dòng và bao nhiêu cột?
10. Những cột nào là dạng số (numeric)?
11. Những cột nào là dạng phân loại (categorical)?
12. Có bao nhiêu tin đăng thuộc từng loại BĐS (Đất thổ cư, Biệt thự, Nhà mặt tiền...)?
13. Giá trung bình của bất động sản là bao nhiêu ?

Làm sạch dữ liệu (Data Cleaning)

14. Kiểm tra các cột có giá trị thiếu (missing values).
15. Chuẩn hóa cột price_text (ví dụ: "3,3 tỷ", "16 triệu/m²") về dạng số.
16. Kiểm tra trùng lặp theo Mã tin.
17. Chuẩn hóa các cột có đơn vị "m", "m²" (Chiều dài, Chiều ngang, area).